Table des matières

Aide-mémoire — Analytics Engineering 1
1. La chaîne, en une ligne 1
2. Les neuf cours 2
3. Les fichiers du projet 3
4. Les commandes 5
5. Les pièges qui reviennent 7
6. Le vocabulaire 7
Comment utiliser cette fiche 10

Aide-mémoire — Analytics Engineering

Formation Analytics Engineering — à garder ouvert à côté des cours

À quoi sert cette fiche. Situer d'un coup d'œil ce que la formation manipule : la chaîne des couches, les neuf cours, les fichiers du projet dbt, les commandes qu'on tape vraiment, les pièges qui reviennent et le vocabulaire. Aucun détail ici : chaque ligne dit où le détail se trouve. Revenez-y dès qu'un mot ou une commande vous bloque. Le cas d'étude est Véhicules Boréal inc., un fabricant fictif de véhicules récréatifs dont l'usine est à Val-des-Aulnes ; Data Engineering charge ses données, et vous en tirez des étoiles.

---

1. La chaîne, en une ligne

`RAW` → `STAGING` → `INTERMEDIATE` → `MARTS` → `PRESENTATION`. Tout vit dans un seul fichier DuckDB, `Projet_Boreal/plateforme/entrepot/boreal.duckdb`, un schéma par couche, et le code qui fabrique les couches vit dans votre dépôt, `Projet_Boreal/02_ae`.

Couche	Ce qu'on y fait	Qui l'écrit	Forme	Préfixe	Cours
`RAW`	les livraisons, telles qu'elles arrivent, tout en texte	Data Engineering, ou le kit 02	tables	aucun	01
`STAGING`	renommer, typer, décoder, tirer l'état	vous, par dbt	vues	`stg_*`	03

© 2026 iAlexMG. Tous droits réservés.

Couche	Ce qu'on y fait	Qui l'écrit	Forme	Préfixe	Cours
	courant ; une vue par table reçue				
`INTERMEDIATE`	la logique qui garde les faits minces	vous	`ephemeral`, aucun objet	`int_*`	04
`MARTS`	les étoiles : dimensions et faits	vous	tables, deux faits en incrémental	`dim_*`, `fct_*`	04 à 07
`PRESENTATION`	un résumé taillé pour un rapport, sans règle de métier neuve	vous	vues	`rpt_*`	05
l'histoire	les versions d'une dimension	vous	un snapshot	`snp_*`	05

Seules les `MARTS` se conçoivent : le Cours 02 dessine les étoiles avant tout outil. Les autres couches s'intercalent sans se concevoir, `STAGING` pour nettoyer, `INTERMEDIATE` pour préparer, `PRESENTATION` pour résumer. Le schéma des séries de l'usine, `argent`, vient aussi de Data Engineering, et se lit comme une source.

Deux environnements, un dépôt. Sur la cible `prod`, les schémas portent leur nom nu ; sur `dev`, la cible par défaut, ils prennent le préfixe `dev_` (Cours 07). Le snapshot fait exception : son schéma, `snapshots`, est fixé par sa configuration et partagé par les deux.

---

2. Les neuf cours

Cours	Titre	Ce qu'il ajoute au projet
00	Générer les données sources	la `RAW` montée, par la voie de Data Engineering ou par le kit de départ
01	Analyser la `RAW` reçue	la `RAW` vérifiée contre la livraison de Data Engineering ; le dépôt `02_ae`
02	Concevoir le modèle dimensionnel	les grains des six processus, la matrice de bus, un prototype en SQL

© 2026 iAlexMG. Tous droits réservés.

Cours	Titre	Ce qu'il ajoute au projet
03	Monter le projet dbt et le `STAGING`	le projet dbt, les sources, les premières vues, l'état courant
04	Bâtir l'étoile des ventes	`dbt_utils`, les clés par hachage, `dim_date`, `fct_ventes` égal à l'ERP
05	Tester, historiser et documenter	les tests, le test unitaire, le snapshot, le type 2, la documentation, la `PRESENTATION`
06	Modéliser les autres faits	l'après-vente et l'usine : accumulation, instantané périodique, fait sans mesure
07	Charger en incrémental	l'incrémental et sa fenêtre de reprise, la télémétrie, les lots de 2026, `dev` et `prod`
08	Mettre en place l'intégration continue	le linteur, le contrat, la fraîcheur, la CI sur GitHub Actions
09	Rebâtir le projet de bout en bout	la documentation sans trou, la reconstruction de zéro, ses trois preuves, l'étiquette `v1.0`

---

3. Les fichiers du projet

Fichier	Où	Ce qu'il fait	Cours
`dbt_project.yml`	racine du dépôt	fait exister le projet et porte ses réglages, en cascade	03
`profiles.yml`	dossier `.dbt` de votre répertoire personnel, jamais au dépôt	dit comment se connecter : le chemin de l'entrepôt, les cibles `dev` et `prod`	03, 07
`packages.yml`, `package-lock.yml`	racine	`dbt_utils`, installé par `dbt deps` à une version fixée	04
`_sources.yml`	`models/staging/`	déclare ce que le projet reçoit et ne modifie pas, et sa fraîcheur	03, 08

© 2026 iAlexMG. Tous droits réservés.

Fichier	Où	Ce qu'il fait	Cours
`_staging.yml`, `_intermediate.yml`	`models/staging/`, `models/intermediate/`	les descriptions des vues et des modèles du milieu	09
`_marts.yml`	`models/marts/`	les descriptions, les tests, le test unitaire et le contrat de l'étoile	05 à 08
`_exposures.yml`	`models/marts/`	ce que Business Intelligence consomme, déclaré dans le graphe	05
un modèle `.sql`	`models/…`	un `select`, que dbt enveloppe en vue ou en table	03
un snapshot `.sql`	`snapshots/`	l'histoire d'une dimension, version par version	05
un seed `.csv`	`seeds/`	une donnée que le projet écrit lui-même, chargée par `dbt seed`	07
un test `.sql`	`tests/`	un test singulier : une requête qui ne doit rendre aucune ligne	05
une macro `.sql`	`macros/`	du Jinja réutilisable : les schémas, le ménage, la garde	03, 08, 09
`.sqlfluff`	racine	la forme du SQL, et le profil du linteur	08
`ci/`, `.github/workflows/ci.yml`	racine	le profil de la CI, sa pile épinglée, la structure vide des sources, le workflow	08
`reconstruire.cmd`	racine	le projet refait de zéro, en neuf commandes	09
`outils/`	racine	la lecture du manifeste, la comparaison de deux entrepôts	08, 09
`LIVRAISON.md`, `README.md`	racine	ce que le projet livre ; par où le reprendre	04 à 09

Le `dbt_project.yml` se lit sur quatre niveaux, et à chaque niveau le mot vient d'ailleurs :

© 2026 iAlexMG. Tous droits réservés.

```yaml
models:              # 1. le type de ressource, un mot de dbt
  boreal_ae:         # 2. le projet, le même que le name: de l'en-tête
    staging:         # 3. un sous-dossier de models/
      +materialized: view    # 4. un réglage, toujours précédé d'un +
      +schema: staging
```

Le + distingue un réglage d'un nom de dossier : oublié, le réglage est ignoré sans un mot. Le réglage le plus précis gagne, du `dbt_project.yml` au fichier de description, puis au `config()` du modèle.

D'où vient la table qu'un modèle lit, et donc comment on l'appelle :

La table de départ est…	Fabriquée par	Déclarée dans	Appelée par
une source	Data Engineering, hors de dbt	`_sources.yml`	`source('raw', 'erp_vbak')`
un seed	`dbt seed`	le dossier `seeds/`, et `dbt_project.yml`	`ref('anomalies')`
un modèle	`dbt run` ou `dbt build`	le dossier `models/`	`ref('stg_commandes')`

---

4. Les commandes

Toutes se tapent dans l'Anaconda Prompt, environnement `formations` actif, depuis `Projet_Boreal/02_ae`, sauf mention contraire.

Commande	Ce qu'elle fait	Cours
`dbt debug`	vérifie le profil, le projet et la connexion	03
`dbt deps`	installe les paquets de `packages.yml`	04
`dbt parse`	relit le projet et réécrit `target/manifest.json`, sans rien construire	08
`dbt compile`	montre le SQL que dbt exécuterait, Jinja résolue	03
`dbt ls --select stg_commandes+`	ce qui dépend d'un modèle ; `+modèle` pour ce dont il dépend	03
`dbt run`	construit les modèles, sans les tests	03

© 2026 iAlexMG. Tous droits réservés.

Commande	Ce qu'elle fait	Cours
`dbt test`	lance les tests ; `--select test_type:singular` pour les seuls tests singuliers	05, 09
`dbt build --target prod --exclude snp_concessionnaire`	construit et teste tout, dans l'ordre du graphe, sauf le snapshot	05, 07
`dbt seed`	charge les seeds	07
`dbt snapshot --target prod --vars "{lot: 2024-06}"`	rejoue un lot de l'histoire des paliers, dans l'ordre	05
`dbt docs generate`	écrit la documentation, le catalogue et le manifeste dans `target/`	05
`dbt source freshness --target prod`	juge l'âge du dernier chargement de chaque source	08
`dbt run-operation nettoyer_dev`	jette les schémas `dev_*` ; refuse la cible `prod`	08
`dbt run-operation exiger_entrepot_neuf --target prod`	refuse un entrepôt qui porte déjà le projet	09
`--full-refresh`	reconstruit un modèle incrémental en entier	07
`--target ci --profiles-dir ci`	construit sur l'entrepôt jetable de la CI	08
`sqlfluff lint models tests snapshots`	vérifie la forme du SQL ; `fix` corrige ce qui se corrige	08
`reconstruire.cmd`	refait le projet de zéro, sur un entrepôt qui n'a que la `RAW` et l'argent	09
`git switch -c capstone`, `git tag -a v1.0`	une branche pour un changement ; une étiquette pour un état livré	08, 09
`duckdb plateforme/entrepot/boreal.duckdb`, depuis `Projet_Boreal`	ouvre l'entrepôt ; `-readonly` pour regarder, `.exit` pour sortir	01

© 2026 iAlexMG. Tous droits réservés.

5. Les pièges qui reviennent

Le symptôme	La cause	Cours
`IO Error: Cannot open file … utilisé par un autre processus`	la ligne de commande DuckDB est restée ouverte : un seul processus à la fois sur l'entrepôt	03
les vues atterrissent dans `staging_staging`	la macro `generate_schema_name` manque	03
`cannot access local variable 'summary_message'`	`profiles.yml` existe mais est vide	03
le chiffre d'affaires sort 5,51 pour cent trop haut	l'état courant n'a pas été tiré, et le lot livré deux fois compte deux fois	03
les mois sortent en anglais	`strftime` et `monthname` rendent l'anglais ; le décodage se fait une fois, dans `dim_date`	02, 04
un `accepted_values` au vert sur une colonne vide	il ignore les valeurs absentes : un `not_null` va toujours à côté	05
des versions qui finissent avant de commencer	les lots du snapshot ont été rejoués dans le désordre, ou par-dessus une histoire	05, 09
un jour de trop ou de moins sur une heure de l'usine	`heure::date` suit le fuseau de la session ; la conversion s'écrit	06
des sorties perdues sans un message	le filtre incrémental porte sur l'événement au lieu de l'arrivée	07
tout est au vert et la production n'a pas bougé	`--target prod` a été oublié : tout est dans `dev_*`	07
`Error: No such option '--profiles-dir'.` au linteur	le profil du linteur se déclare dans `.sqlfluff`, sous `[sqlfluff:templater:dbt]`	08
une comparaison de vues qui rend toujours 0	une vue nomme sa base ; celle d'un entrepôt attaché lit les tables de l'autre	09

---

6. Le vocabulaire

© 2026 iAlexMG. Tous droits réservés.

Terme	En une phrase
Entrepôt de données	une base pensée pour analyser, et non pour faire tourner une application ; ici, un fichier DuckDB
OLTP et OLAP	le traitement transactionnel en ligne, celui de l'ERP qui enregistre une commande, contre le traitement analytique en ligne, celui qui les compte par millions ; la formation fait le second
ELT	charger d'abord, transformer ensuite, dans l'entrepôt : Data Engineering charge, vous transformez
dbt	un logiciel en ligne de commande qui compile du SQL et du Jinja, l'exécute dans l'ordre et le teste ; le « T » de l'ELT
Modèle	un fichier `.sql` qui contient un `select`, et que dbt matérialise
Matérialisation	la forme que prend un modèle : `view`, `table`, `ephemeral`, `incremental`
Source	une table que le projet reçoit et ne fabrique pas, appelée par `source()`
`ref()`	l'appel d'un modèle ou d'un seed ; c'est lui qui trace le graphe
DAG	le graphe orienté acyclique des dépendances, que dbt tire des `ref()` et des `source()` et qu'il parcourt dans l'ordre
État courant	la dernière version de chaque clé, tirée d'une source qui garde toutes ses livraisons
Grain	ce qu'une ligne d'un fait représente, déclaré avant les colonnes
Fait, dimension, étoile	les événements chiffrés au centre, leur contexte autour, reliés par des clés
Fait de transaction, d'accumulation, instantané périodique, sans mesure	un événement ; un dossier qui franchit des jalons ; un état à intervalle régulier ; un événement qui n'a que ses clés
Clé de substitution	un identifiant fabriqué par l'entrepôt, ici par hachage, que le fait porte à la place de la clé de la source
Dimension conforme	une dimension partagée par plusieurs faits, définie une fois ; c'est elle qui permet le forage transversal

© 2026 iAlexMG. Tous droits réservés.

Terme	En une phrase
Dimension dégénérée	une clé que le fait porte sans dimension derrière elle, comme un numéro de commande
Matrice de bus	le tableau des processus contre les dimensions conformes qu'ils partagent
Type 1, type 2	écraser l'ancienne valeur d'un attribut, ou garder une version par valeur, datée
Snapshot	le mécanisme de dbt qui garde les versions d'une source ; la seule table du projet qui porte une histoire
Seed	un petit fichier CSV versionné, chargé par `dbt seed`
Test générique, singulier, unitaire	un test déclaré en YAML sur une colonne ; une requête qui ne doit rien rendre ; un modèle essayé sur des entrées choisies
Témoin indépendant	un test qui compare l'étoile à un chemin qui ne passe pas par elle, l'ERP ou un compteur d'usine
Contrat	les colonnes et les types qu'un modèle promet, vérifiés avant de créer la table
`exposure`	la déclaration, dans le projet, de ce qu'un consommateur lit
Incrémental	un modèle qui n'ajoute que ce qui est arrivé depuis la dernière exécution, sur une clé unique
Fenêtre de reprise	les jours qu'un modèle incrémental relit à chaque passe, pour rattraper les retards
Fraîcheur	l'âge du dernier chargement d'une source, réglé sur sa cadence
`manifest.json`	le catalogue que dbt écrit : modèles, colonnes, descriptions, tests, graphe ; ce qu'un agent lit
CI	l'intégration continue : rejouer les vérifications à chaque changement proposé, sur une machine neuve
Linteur	l'outil qui vérifie la forme du code, ici sqlfluff

© 2026 iAlexMG. Tous droits réservés.

Terme	En une phrase
Reconstruction	le projet refait de zéro depuis son dépôt, sur un entrepôt qui n’a que ce qu’il reçoit
Garde	une vérification qui refuse de lancer une opération au mauvais endroit

---

Comment utiliser cette fiche

- Perdu sur « où on va » : section 1.
- Un cours à situer : section 2.
- Un fichier du projet vous surprend : section 3.
- Une commande à retrouver : section 4.
- Un message d’erreur ou un chiffre qui ne tombe pas : section 5.
- Un mot qui bloque : section 6.

© 2026 iAlexMG. Tous droits réservés.